

Design, Implementation, and Evaluation of a Neural-Network-Based Quadcopter UAV System

Fan Jiang , Member, IEEE, Farhad Pourpanah , Member, IEEE, and Qi Hao , Member, IEEE

Abstract—In this paper, a quadcopter unmanned aerial vehicle (UAV) system based on neural-network enhanced dynamic inversion control is proposed for multiple real-world application scenarios. A sigma-pi neural network (SPNN) is used as the compensator to reduce the model error and improve the system performance in the presence of the uncertainties of UAV dynamics, payload, and environment. Besides, we present a technical framework for fast and robust implementation of multipurpose UAV systems and develop a testbed for the evaluation of UAV control system by using a high-precision optical motion capture system. Both simulation results and experiment results demonstrate that the SPNN can reduce the inversion errors related to UAV parameter uncertainties as well as tracking errors related to unknown disturbances and unmodeled dynamics. With the help of an online neural network (NN) learning mechanism, the entire system can achieve much higher accuracy in attitude and trajectory control than that achieved by conventional proportional-integral derivative based control systems under varying flight conditions.

Index Terms—Unmanned aerial vehicles (UAVs), control systems, neural network applications.

I. INTRODUCTION

UNMANNED aerial vehicles (UAVs) have been widely used in many applications, such as surveillance and reconnaissance, power-line inspection, precision agriculture, environment monitoring, as well as rescue and disaster management. Recently, multirotor UAVs have attracted lots of industrial attention. Main advantages of multirotor UAVs include low cost, simple structure, high maneuverability, vertical take-off and landing ability, long loiter time, and easy maintenance [1], [2]. However, developing long-range, high-payload, all-weather,

industrial-grade multirotor UAVs requires extensive innovations in material, structure, motors and sensors, operating system, and, especially, flight control algorithms.

To develop a robust flight controller, several issues must be taken into account, such as external disturbances, delays in processing and communication, and actuator degradation [3]. Most quadcopters employ a two-stage controller scheme, with the inner loop controlling the attitude and the outer loop controlling the position. This allows the stabilization of the aircraft without requiring a position feedback. Various control mechanisms have been developed for the position and attitude control of multirotor UAVs, from proportional-integral derivative (PID) control [4] to linear-quadratic regulators (LQRs) [5], dynamic inversion [6], fuzzy control [7], backstepping and sliding-mode control [8], just to name a few. To control the underactuated, multi-input, and strong-coupling quadrotor UAV systems, conventional controllers have to be incorporated with intelligent tuning schemes to achieve the balance between robustness and adaptability of the system. For example, PID control algorithms have been extensively used in UAV control systems because of their simple implementation and flexible tuning. However, they suffer from many drawbacks as follows.

- 1) The PID controller is a linear and low-bandwidth control system, which cannot adapt well to highly nonlinear systems with strong, high-frequency disturbances, while the multirotor UAV platform is nonlinear and further complicated by flight tasks and the changing environment [9], [10].
- 2) The dynamics of sensors and actuators are usually neglected in the control design [4], which causes problems in real-world implementation, e.g., delay in response, inaccuracy in following the command.
- 3) There are a variety of uncertainties such as loading variations, external disturbances, and changing plant parameters, which classic PID control techniques cannot handle well.

Dynamic inversion is an effective approach to control both linear and nonlinear systems by directly reducing the complexity of the system dynamics [6]. The method adapts well to the environmental changes and, hence, is suitable for high maneuverability control problems [11]. However, because of its high sensitivity to the variations of plant parameters, a complementary feedback control mechanism is usually required to improve the system robustness and stability. The inclusion of artificial neural networks (ANNs) can solve this problem [12], [13]. ANNs are able to reduce the inversion error [14] and

Manuscript received June 28, 2018; revised October 31, 2018 and January 1, 2019; accepted February 14, 2019. Date of publication March 28, 2019; date of current version October 31, 2019. This work was supported in part by the National Natural Science Foundation of China under Grant 61773197, in part by the Science and Technology Innovation Commission of Shenzhen under Grant CKFW2016041415372174 and Grant GJHZ20170314114424152, and in part by the Nanshan District Science and Technology Innovation Bureau under Grant LHDT20170007. (Corresponding author: Qi Hao.)

F. Jiang and Q. Hao are with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen 518055, China (e-mail: jiangf@sustc.edu.cn; hao.q@sustc.edu.cn).

F. Pourpanah is with the College of Mathematics and Statistics, Shenzhen University (SZU), Shenzhen 518060, China (e-mail: farhad.086@gmail.com).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TIE.2019.2905808

compensate various uncertainties [10], despite many implementation issues.

The integration of ANNs into multirotor UAV control systems can be classified into two groups: 1) neural network (NN)-based estimator/compensator [12], [15], [16] and 2) NN-based (either adaptive or hierarchical) full-state feedback [17]. NN-based estimators/compensators work as an adaptive estimator, which estimate strong disturbance models such as plant variations, gusts, or periodical disturbances, and then utilize closed-loop feedforward output to cancel these modeled disturbances. Such a method does not change the stability of the control system and improves the system performance without requiring labor-consuming parameter tuning process.

Because of the intrinsic multimodalities of multirotor UAVs, Gaussian potential function networks [10], fuzzy NNs [18], [19], and dual NNs [20] have been employed to control multivariable nonlinear flight systems. Under these schemes, both global and local optimization are used to achieve the tradeoff between stability margin and optimal performance: The global search for optimal control parameters can be performed by using offline random optimization methods such as particle swarm optimization [18]; local control parameter optimizations are performed by using online gradient based methods to achieve fast convergence [10], [19].

Several types of ANNs have been used to adaptively compensate dynamic inversion errors, such as single hidden layer neural networks [21], multilayer perceptrons [22], fuzzy-based neural networks [23], and sigma-pi neural networks (SPNNs) [24]. Among them, SPNNs are popular owing to their simple structure and fast convergence for online adaptation [3], [13], [25], [26]. A comparison of existing control methods with SPNN is presented in Table I. SPNNs are advantageous in that they can compensate the inversion errors and external disturbances by adjusting the weights in real time by using a simple control law with guaranteed Lyapunov convergence for bounded inputs [14], [24]. Despite the superior theoretical advantages, most of the works regarding SPNN-based control of multirotors are of simulation, and little work has been done with the real-world implementation and benchmarking for multirotor UAVs, because of the shortage of computing power of existing open-source controllers, the complexity associated with real-time neural network implementations, and many other technical challenges.

In this paper, an SPNN-augmented dynamic inversion controller is proposed to improve the flight control performance of UAVs. The robust performance of the proposed control methods are verified by using hardware-in-the-loop (HITL) simulations and real-world experiments evaluated by a high-precision optical motion capture system. To further simplify the computational complexity of the real-time implementation within the UAV, extensive software optimizations are employed to the initialization of nonlinear base functions and the feedback tuning of ANN weights, which turns out to achieve satisfactory performance both in simulations and experiments.

The major contributions of this research are as follows:

- 1) developing an efficient SPNN-based approach to minimize dynamic inversion error and handle external disturbances and uncertainties;

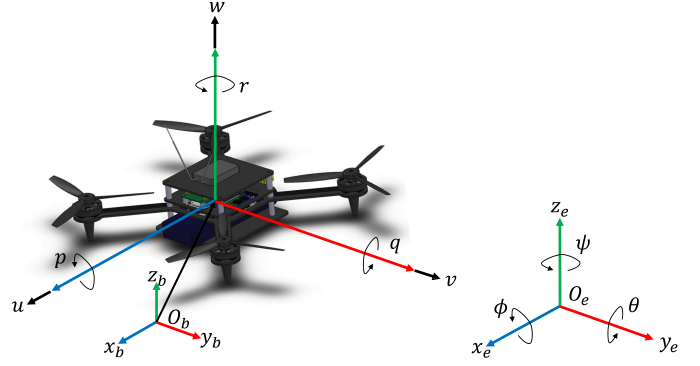


Fig. 1. Basic model of quadrotor UAVs.

- 2) developing a systematic framework for adaptive flight control of quadrotor UAVs;
- 3) developing a systematic framework for quadrotor UAV system implementations with real-world applications;
- 4) developing a set of hardware and software tools to help implement and fine-tune the control algorithm in low-cost, commercially available platforms;
- 5) developing a reliable multirotor UAV system evaluation environment for benchmarking performance metrics.

The remaining of this paper is organized as follows. Section II sets up the whole control system and states the control problem. Sections III and IV give the details of the proposed NN-based dynamic inversion control system. Section V provides both simulation and experimental results, and gives discussions. Section VI concludes this paper.

II. SYSTEM SETUP AND PROBLEM STATEMENT

In this section, the dynamics of quadrotor UAVs is explained first, and then the structure of conventional PID control is described in detail; finally, the problem statement is presented.

A. Quadrotor Dynamics

As shown in Fig. 1, a quadrotor UAV is an underactuated system with six degrees of freedom (DOFs) [27], where $E = \{x_e, y_e, z_e\}$ and $B = \{x_b, y_b, z_b\}$ indicate inertial and body frames, respectively. Furthermore, $\mathbf{x} = (x, y, z)^T$ represents the position and $\mathbf{V} = (u, v, w)^T$ represents the velocity of the quadrotor. Similarly, $\Theta = (\phi, \theta, \psi)^T$ and $\omega = (p, q, r)^T$ indicate the attitude and rotational velocity of the quadrotor, respectively. The rotation matrix, $\mathbf{C}_b^e$, for transforming coordinates from E to B is given as follows [28]:

$$\mathbf{C}_b^e = \begin{bmatrix} c\theta c\psi - s\phi s\psi s\theta & -c\phi s\psi & c\psi s\theta + c\theta s\phi s\psi \\ c\theta s\psi + c\psi s\phi s\theta & c\phi c\psi & s\psi s\theta - c\psi c\theta s\phi \\ -c\phi s\theta & s\phi & c\phi c\theta \end{bmatrix} \quad (1)$$

where $s(\cdot)$ and $c(\cdot)$ denote $\sin(\cdot)$ and $\cos(\cdot)$, respectively. The angular velocity of the UAV in the body frame is related to the derivatives of the roll, pitch, and yaw angles according to

TABLE I
COMPARISON OF CONTROL SCHEMES IN QUADROTORs

Characteristics	Robust Adaptive Precision			Disturbance Rejection	Implementation Difficulty	Notes
PID	Yes	No	+	+	-	A low-order controller, model-free, easy to implement, incapable of dealing with high-order dynamics
LQR	No	No	++	-	+	A high-order controller, model-based, easy to design and hard to implement, incapable of dealing with uncertainties
H_∞	Yes	Yes	++	++	++	A robust high-order controller, model-based, hard to design and implement, incapable of adaptive tuning
Sliding Mode	Yes	Yes	++	+++	++	A nonlinear robust controller, model-free, hard to design a suitable sliding mode
Backstepping	Yes	Yes*	++	++	++	A robust nonlinear controller, model-based, high computational-complexity to implement, incapable of dealing with disturbances
MPC	Cond.	No	+++	++	+++	A high-order open-loop control, model-based, high computational-complexity; sensitive to uncertainties
Dynamic Inversion	No	No	++	-	+	A open-loop compensator, model-based, easy to design and implement, sensitivity to uncertainties and disturbances
DI with SPNN	Yes	Yes	++	++	+	A robust adaptive compensator, partly model-based, easy to design and implement, capable of rejecting disturbances and model errors

the following [28]:

$$\begin{bmatrix} \ddot{p} \\ \ddot{q} \\ \ddot{r} \end{bmatrix} = \begin{bmatrix} c\theta & 0 & -c\phi s\theta \\ 0 & 1 & s\phi \\ s\phi & 0 & c\phi c\theta \end{bmatrix} \begin{bmatrix} \ddot{\phi} \\ \ddot{\theta} \\ \ddot{\psi} \end{bmatrix}. \quad (2)$$

According to Newton's second law, the transitional dynamics equation of the center of mass (CoM) of the UAV can be written as [28]

$$\frac{d(mv)}{dt} = \vec{F} \quad (3)$$

where m is the mass of the UAV and $\vec{F}$ the total force, which contains gravity (F_G), drag force (F_D), and forces from all the rotors (F_T). The drag force can be written as

$$F_D = -k_d \dot{\mathbf{x}} \quad (4)$$

where k_d are the drag coefficients. The force of each rotor can be described as

$$F_i = k_F w_i^2 \quad (5)$$

where w_i is the angular speed of the i th rotor and k_F a coefficient that can be adjusted by experiments or simulations. The total force from rotors, which is in the z -direction, can be determined as follows:

$$F_T^b = [0 \quad 0 \quad k(w_1^2 + w_2^2 + w_3^2 + w_4^2)]^T \quad (6)$$

where k is lift coefficient of the rotor. In the inertial frame, the lift force can be written as

$$F_T = C_b^e F_T^b. \quad (7)$$

And, the motion can be written as

$$m\ddot{\mathbf{x}} = [0 \quad 0 \quad -mg]^T + F_T + F_D \quad (8)$$

where g is the gravitational acceleration constant. In addition to force, each rotor generates a vertical moment to the plane rotation of propellers, which is

$$M_i = k_M w_i^2 \quad (9)$$

where k_M is the moment constant. Two rotors, i.e., 1 and 3, rotate in the z_b -direction, while the other two rotors, i.e., 2 and

4, rotate in the z_b -direction. Thus, the angular acceleration using Euler equations can be written as follows [28]:

$$I \begin{bmatrix} \ddot{p} \\ \ddot{q} \\ \ddot{r} \end{bmatrix} = I \begin{bmatrix} L(F_2 - F_4) \\ L(F_3 - F_1) \\ M_1 - M_2 + M_3 - M_4 \end{bmatrix} - \begin{bmatrix} p \\ q \\ r \end{bmatrix} \times I \begin{bmatrix} p \\ q \\ r \end{bmatrix} \quad (10)$$

which can be written as follows [28]:

$$I \begin{bmatrix} \ddot{p} \\ \ddot{q} \\ \ddot{r} \end{bmatrix} = I \begin{bmatrix} 0 & L & 0 & -L \\ -L & 0 & L & 0 \\ \gamma & -\gamma & \gamma & -\gamma \end{bmatrix} \begin{bmatrix} F_1 \\ F_2 \\ F_3 \\ F_4 \end{bmatrix} - \begin{bmatrix} p \\ q \\ r \end{bmatrix} \times I \begin{bmatrix} p \\ q \\ r \end{bmatrix} \quad (11)$$

where $\gamma = \frac{k_M}{k_F}$ and L is the distance between the CoM and the rotors.

Note that for quadcopter frames with alternative motor configurations (X-shaped, etc.), it is always possible to map the motor thrust to fit the form shown in (11).

B. Conventional PID Control

As shown in Fig. 2, the control system of the UAV contains two loops: 1) inner loop and 2) outer loop. These two loops are described as follows.

Attitude controller (inner loop): This loop controls the attitude of the UAV, and the relationship between the angular acceleration and the torque acting on the UAV can be described as

$$\ddot{\Theta} = \mathbf{M}_\tau / \mathbf{I}. \quad (12)$$

The pseudocontrol can be written as [13]

$$\mathbf{U}_\tau = \mathbf{K}_p(\Theta_d - \Theta) + \mathbf{K}_d(\dot{\Theta}_d - \dot{\Theta}) + \mathbf{K}_i \int (\Theta_d - \Theta) d\Theta \quad (13)$$

where attitude Θ_d is the desired angular attitude, and Θ the output of the plant dynamics, which can be measured by sensors.

Position controller (outer loop): This loop controls the position of the UAV. The relationship between the acceleration of the CoM and the force acting on the UAV can be described as

$$\ddot{\mathbf{x}} = \mathbf{F}_T / m. \quad (14)$$

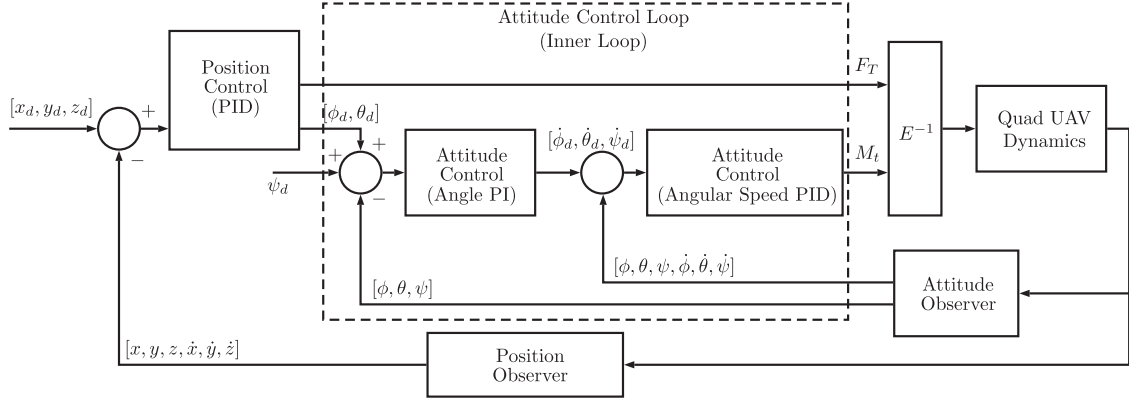


Fig. 2. PID-based UAV attitude control loop.

Similarly, the pseudocontrol can be written as [13]

$$\mathbf{U}_T = \mathbf{K}_p(\mathbf{x}_d - \mathbf{x}) + \mathbf{K}_d(\dot{\mathbf{x}}_d - \dot{\mathbf{x}}) + \mathbf{K}_i \int (\mathbf{x}_d - \mathbf{x}) d\mathbf{x}. \quad (15)$$

Then, the desired values of ϕ and θ are given as follows:

$$\begin{aligned} \phi_d &= \arcsin\left(\frac{m}{T}(s\psi_d U_x - c\psi_d U_y)\right) \\ \theta_d &= \arcsin\left(\frac{(U_x m - T s\psi_d s\phi_d)}{T c\psi_d c\phi_d}\right) \end{aligned} \quad (16)$$

where

$$T = m\sqrt{U_x^2 + U_y^2 + (U_z + g)^2}.$$

C. Problem Statement

In this paper, to design and implement robust control systems for quadrotor UAVs, the following technical challenges have to be addressed.

- 1) How to design a dynamic inversion model to improve the performance of PID controllers.
- 2) How to integrate an SPNN with the dynamic inversion model to enhance the performance of the controller against unmodeled dynamics, parameter mismatch, and wind perturbations.
- 3) How to implement the SPNN-based dynamic inversion control system in real-world applications.

III. NEURAL-NETWORK-BASED DYNAMIC INVERSION FLIGHT CONTROL

In this section, the structures of the dynamic inversion control system and proposed NN-based dynamic inversion models are described in detail.

A. Dynamic Inversion

Consider the UAV control system as a nonlinear control system with n DOFs, which can be described as

$$\ddot{\mathbf{x}} = f(\mathbf{x}, \dot{\mathbf{x}}, \delta) \quad (17)$$

where $\mathbf{x}$ and $\dot{\mathbf{x}}$ are state variables, δ the control variable, and f maps the control variable to the state variables. It is convenient to define the pseudocontrol variable $U = \ddot{\mathbf{x}}$. So

$$U = f(\mathbf{x}, \dot{\mathbf{x}}, \delta). \quad (18)$$

If f is an invertible mapping with δ , then

$$\delta = f^{-1}(\mathbf{x}, \dot{\mathbf{x}}, U). \quad (19)$$

If f describes the model of the quadrotor, the inverse of f is precisely equal to the control variable. In fact, f is an analytical model for the dynamics of the quadrotor. To offset the analytical model of the quadrotor, $\hat{\delta}$ can be defined as

$$\hat{\delta} = \hat{f}^{-1}(\mathbf{x}, \dot{\mathbf{x}}, U). \quad (20)$$

Then, the control system of quadrotor can be described as

$$\ddot{\mathbf{x}} = U + \Delta(\mathbf{x}, \dot{\mathbf{x}}, \delta) \quad (21)$$

where

$$\Delta(\mathbf{x}, \dot{\mathbf{x}}, \delta) = f(\mathbf{x}, \dot{\mathbf{x}}, \delta) - \hat{f}^{-1}(\mathbf{x}, \dot{\mathbf{x}}, U)$$

and Δ represents the inversion error.

B. Neural Network Augmented Dynamic Inversion

Since the approximate dynamic model of quadrotor has been deduced in Section III, the inversion model can be used to linearize the nonlinear system and guarantee the stability and reliability of the system. Moreover, to eliminate the error caused by the dynamics model, a neural network is added to adaptively compensate the inversion error online (see Fig. 3). We have the following:

$$U = U_{\text{PID}} + U_d - U_{\text{NN}} \quad (22)$$

where U_{PID} is the PID pseudo control of the inner loop, $U_d = \ddot{\mathbf{x}}_d$ the desired acceleration vector, and U_{NN} the output of the NN to cancel the inversion error Δ . We have the following:

$$U_{\text{NN}} = W^T \beta(\mathbf{X}, U) \quad (23)$$

where W is the weights of the NN and β the basis function. By substituting (22) into (21), and setting $K_I = 0$, the system can

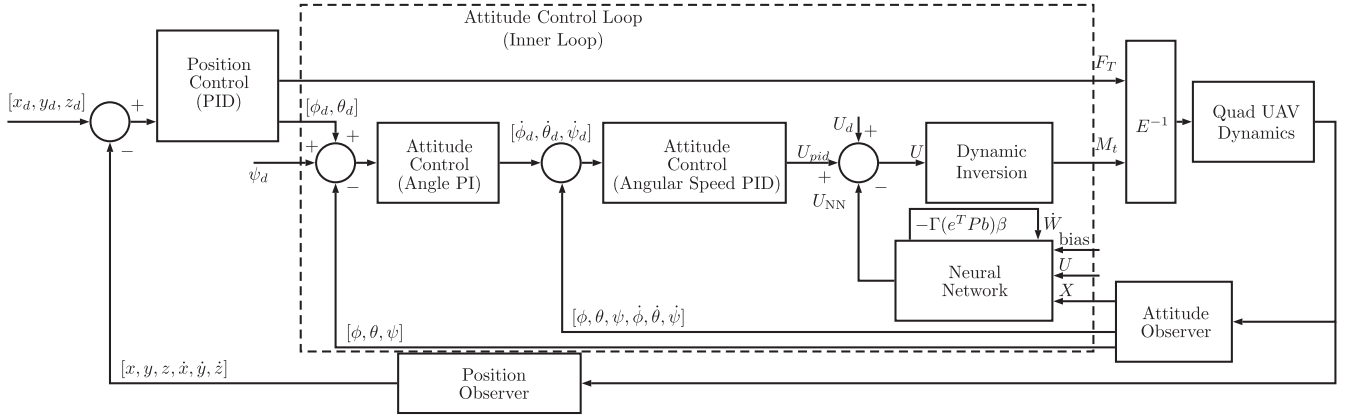


Fig. 3. NN compensator based UAV attitude control loop.

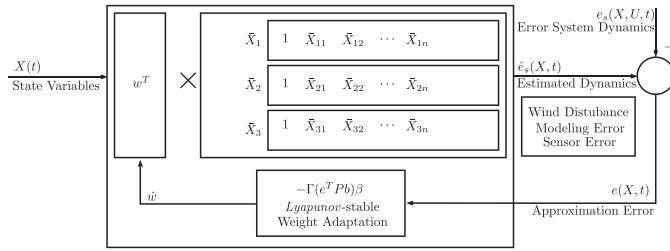


Fig. 4. Structure of an SPNN.

Algorithm 1: The Neural Network Adaptation.

Input: State X , Pseudo control U , Weights W , Set of bases $\beta(X, U)$

Output: NN output U_{NN}

- 1: $W = W_0 = \mathbf{0} + \sigma$ ▷ Initialization
- 2: **while** System is normal **do**
- 3: Get next states X, U
- 4: $U_{NN} = W^T \beta(X, U)$
- 5: $\dot{W} = -\Gamma(e^T P b) \beta$ ▷ Weight feedback
- 6: $W = W + \dot{W} \cdot \Delta t$ ▷ Adjust for step time
- 7: **end while**

be described as

$$\ddot{\tilde{x}} + K_P(\tilde{x}) + K_D(\dot{\tilde{x}}) = U_{NN} - \Delta \quad (24)$$

where $\tilde{x} = \ddot{x}_d - \ddot{x}$.

The error vector in the pitch channel can be written as

$$\dot{e} = Ae + b(U_{NN} - \Delta_\theta) \quad (25)$$

where $b = [0 \ 1]^T$, $e = [\tilde{\theta} \ \dot{\tilde{\theta}}]$ and

$$A = \begin{bmatrix} 0 & 1 \\ -K_P & -K_D \end{bmatrix}. \quad (26)$$

The corresponding weight error can be described as

$$\tilde{W} = W - W^* \quad (27)$$

where W^* is the ideal weight. The error vector can be written as

$$\dot{e} = Ae + b\tilde{W}^T \beta + b(W^{*T} \beta - \Delta_\theta). \quad (28)$$

Similar results can be obtained for roll and yaw channels using the same approach.

C. Sigma-Pi Neural Network

The structure of the single-layer SPNN, shown in Fig. 4, can be written as

$$U_{NN} = \sum_{i=1}^n w_i \beta_i(\bar{X}, v) = W^T \beta(\bar{X}, v) \quad (29)$$

where $\bar{X}$ denotes the normalized states, w the network weights, and β a vector of the basis function composed of a combination of elements $\bar{X}_1, \bar{X}_2, \bar{X}_3$ by means of the Kronecker product, that is

$$\beta = \text{kron}(\text{kron}(\bar{X}_1, \bar{X}_2), \bar{X}_3). \quad (30)$$

The update law for the NN can be written as

$$\dot{W} = \dot{W} = -\Gamma(e^T P b) \beta \quad (31)$$

where the learning rate $\Gamma = \Gamma^T > 0$ and

$$P = \begin{bmatrix} \frac{K_P}{K_P} + \frac{1}{2K_D} & \frac{1}{2K_P} \\ \frac{1}{2K_P} & \frac{1+K_P}{2K_P K_D} \end{bmatrix} \quad (32)$$

where P is the positive matrix in the Lyapunov equation for the linear error system with the NN weights [12].

Note that the same NN is applied to each of the channels (roll, pitch, and yaw) with different weights w^T .

For parallel augmentation tasks, the initial weight vector is set to $W_0 = \mathbf{0} + \sigma$, where $|\sigma| \ll |\det(P)|$.

The online learning process is illustrated in Algorithm 1, whose time complexity is $O(N)$, where $N = |\bar{X}_1| \times |\bar{X}_2| \times |\bar{X}_3|$ is the number of neurons.

D. Tuning of Hyperparameters

The stability and performance of the SPNN are closely tied with the learning rate Γ . However, Γ is only a max-bounded

TABLE II
DESIGN GOALS AND CHOSEN PARAMETERS FOR THE QUADCOPTER

Specifications		Design Parameters	
Max Speed	30 km/h	Motor Power	300 W
Max Payload	1 kg	Weight	360 g
Outdoor Range	2 km	Supply Voltage	12 V
Diagonal Size	260 mm	Flight Controller	Single-chip
Max Swarm Size	30	Local Mesh Comm	AP6255
Max Angular Error	0.05 rad	Long Range Comm	RT3572
Cost	\$60	Body Thickness	6 mm

Algorithm 2: Searching for the Best Learning Rate.

Input: Upper bound of learning rate $\bar{\Gamma}$, Precision σ

Output: Optimal learning rate $\hat{\Gamma}$

Require: $\bar{\Gamma} \geq \hat{\Gamma}, \sigma > 0$

```

1: while  $\bar{\Gamma} > \sigma$  do
2:    $\tilde{\Gamma} = \frac{1}{2}\bar{\Gamma}_k$ 
3:   if  $\tilde{\Gamma}$  is successful in HITL test then
4:      $\Gamma_{k+1} = \frac{\bar{\Gamma} + \Gamma_k}{2}$ 
5:   else
6:      $\Gamma_{k+1} = \tilde{\Gamma}$ 
7:   end if
8:    $\dot{\Gamma} = |\Gamma_{k+1} - \Gamma_k|$ 
9: end while
10:  $\hat{\Gamma} = \Gamma_{k+1}$ 

```

arbitrary unit-less number that is dependent on hard-to-determine factors like the maximum allowed tracking error and PID coefficients [14], and we discovered that the system is unstable if we simply use the theoretical maximum bound. To tackle this problem, a parameter search is performed using our HITL simulator to find the optimal value for Γ . Namely, the simulated system is asked to perform a complex flight trajectory with a descending Γ , and the learning rate with the best flight envelope is chosen. The algorithm is presented as Algorithm 2.

IV. SYSTEM DESIGN AND IMPLEMENTATION

Many technical issues have to be resolved for engineering a platform that can run the proposed sophisticated algorithms in real time. For a realistic flight control system, the following engineering goals should be achieved:

- 1) the ability to autonomously carry out missions in the real world with real-time telemetry;
- 2) the ability to work in dense formations with stable communication links;
- 3) the ability to agilely adapt both to indoor and outdoor environments;
- 4) size, weight, power, and cost.

As an example, we designed a fully customizable industrial grade UAV system, whose system specifications and design parameters are shown in Table II. Besides, we developed a single-chip integrated flight controller system (FCS), called BFLIGHT, as shown in Fig. 5.

The design of the UAV itself involves many aspects that contradict one another. In fact, the developed algorithms, related to

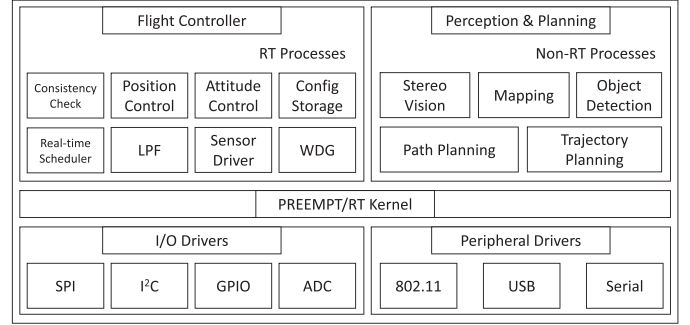


Fig. 5. System block diagram of the multirotor UAV control and computation platform. RT: Real-time. LPF: Low-pass filter. WDG: Watchdog.

hardware and software, are all for fulfilling the design specifications. For example, the reason to develop a COTS SoC-based single-chip flight controller is that we need to minimize the cost and the weight to make space for the payload.

Structural stability is an important factor in the design of a quadcopter UAV. It depends both on the mechanical design and the material used. In our design process, we used finite-element analysis (FEA) to simulate the UAV structure under different values of motor power to ensure the integrity of the vehicle framework. However, FEA cannot replace real-world experiments as commercially available materials usually have different properties than those of the materials used in the simulation. For example, the FEA-validated body thickness of 3 mm is proved to be invalid in our flight tests.

Conventionally, the design of UAVs usually employs a loosely coupled approach, i.e., the flight controller and the perception computer are separate. Meanwhile, in our system, the two systems are tightly coupled, running simultaneously in the same processor. This significantly reduced the hardware complexity and production costs of the system. It also allows all the software components to communicate with the same protocol and middleware, like the robotics operating system (ROS), which makes the development of application software much easier. However, this approach also imposes stringent requirements on the system timing and task scheduling.

To resolve the timing issue, we separated the system tasks into two categories: 1) real-time (RT) and 2) non-real-time (non-RT) (see Fig. 5). We also ported the PREEMPT/RT real-time Linux operating system to our platform, which allows a scheduling latency of $\leq 100 \mu s$. For the RT tasks, we put them at real-time priority, allowing them to preempt the Linux kernel to minimize latency. For non-RT tasks, we carefully tweak the process priorities to prioritize more critical tasks such as object detection and avoidance.

It is also worthwhile to remark that commonly available consumer-oriented application processors, such as the Allwinner H3 we chose, have a dynamic voltage and frequency scaling (DVFS) system to save power, which is useful for many consumer electronics applications but not for our case. The DVFS function must be turned OFF as the kernel will lock all the cores when switching the frequency, causing large unpredictable latencies.

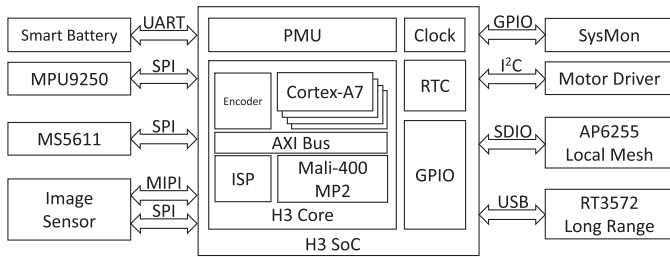


Fig. 6. System hardware architecture.

TABLE III
PRIORITIES IN THE FLIGHT CONTROLLER SOFTWARE

Task	Priority	Real-time	Remarks
Sensors	+++	RT	Minimal Flyable
EKF	++	RT	Required for Precision
SPNN	+++	RT	Minimal Flyable
Position Control	++	RT	Required for Autonomy
Telemetry	+		Required for Monitoring
Remote Control	+++	RT	Minimal Flyable
Health Check	+		
Visual Odometry	+		Best-Effort
Video Downlink	++		Required for Long Range

An overview of the hardware architecture is shown in Fig. 6. The FCS is implemented on an Allwinner Technology H3 SoC, which has 4 ARM Cortex-A7 cores running at 1.2 GHz, with a total power consumption of 2 W. The onboard IMU is an InvenSense MPU-9250, which has an internal Asahi Kasei AK8963 magnetometer. There is also a MEAS Switzerland MS5611 barometer onboard. Also, the airframe is in-house designed and manufactured using carbon fiber. The propulsion system is a set of DJI Snail quadcopter motor system.

Thanks to the one-chip design, the flight software stack is written entirely with the ROS as middleware, enabling full portability between simulation and the real world, greatly reducing development and testing cost.

Sophisticated software optimization has been employed to meet the strict timing requirements of the flight control task. For example, the feedback tuning loop is written in pure C++ with ARM NEON intrinsics to fully utilize the vector processing capability of the Cortex-A7 core and minimize the processing time. Before the optimization, for each axis we have about 2000 instructions for base generation, 1000 for the calculation of output, and 2000 for weight feedback. By performing the optimization by using SIMD, the number of instructions is reduced by ten times, totaling to about 500 instructions. All state variables are estimated using an EKF parallel to the control loop. Together with other minor optimizations, the NN-based attitude control loop is able to run at 1 kHz with a less than 1 ms delay. This ensures optimal performance in challenging conditions such as turbulence and heavy noise.

Besides, the task priorities in the flight controller are carefully optimized to avoid catastrophic failures during test and development procedures. In other words, the “minimal flyable” form of the flight controller is always kept regardless of the failure of other components. The optimized priorities of the tasks are shown in Table III.

As ROS is inherently flawed in real-time sensitive applications with its lockup-prone design, no ROS functions are used in the core flight controller code. Furthermore, thanks to our one-stack architecture, all timing violations are checked and fixed before flight with real-time tracing in HITL simulation.

Real-time communication between drones and ground assets has long been a difficult problem. Conventional telemetry systems such as ZigBee, 915/433 MHz digital modem, and normal Wi-Fi all have distinct strengths and limitations, such as limited bandwidth, high interference, as well as regulatory issues. In our system, we designed a 802.11n based mesh network system to handle both indoor and outdoor communications, which has much more throughput and network capacity than those of conventional systems. A detailed comparison is shown in Table IV.

V. RESULTS AND DISCUSSIONS

In order to validate the effectiveness of the proposed SPNN-based dynamic inversion control systems, a set of performance evaluations has been conducted. Two control schemes, both conventional PID and the proposed SPNN-based dynamic inversion control systems, have been simulated using a simulator and then implemented in a real quadrotor UAV, respectively. Note that because of the consistent software architecture, both the simulator and the real UAV hardware run exactly the same algorithm.

A. Simulation Results

In this section, PID and two types of SPNN-augmented dynamic inversion control systems, namely the raw unoptimized version in [3] and our version, were implemented and simulated in software. The nominal parameters and the initial conditions of the quadrotor for simulation are listed in Table V.

In the simulation, the vehicle was ordered to execute a preset flight plan (see Fig. 8). At each waypoint, the vehicle will stop for 5 s. The nominal airspeed is set to be 5 m/s. To reduce the disparity between simulation and reality, Gaussian noise is added to the simulated sensor readings. The flight trajectory is shown in Fig. 8. The trajectory error is calculated from the shortest distance between the vehicle’s positions and the preset flight trajectory. The calculated error is shown in Fig. 9.

It can be clearly seen that our NN-based control system outperformed both the PID-based control system and the system in [3] in terms of trajectory accuracy. Also, our controller completes the mission with the shortest mission time.

B. Experiment Setting

In order to benchmark the real-world performance of the proposed control scheme with highly variable payload, the drone is ordered to do a simple hovering task with a dangling, long, heavy tether (Fig. 10). The tether we use is rated as American Wire Gauge 12 (29.9 g/m), with a total weight of 104.7 g at hovering altitude (1.5 m). Note that the vehicle itself only weighs 360 g. The test arena is indoor and does not have wind disturbance. The major source of disturbances here are: 1) the off-center,

TABLE IV
COMPARISON OF DIFFERENT UAV COMMUNICATION METHODS

Name	Bandwidth	Capacity	Power	In/Outdoor	Mesh	Remarks
ZigBee	~100 Kbps	~100s	~10 mW	Both	Yes	Susceptible to congestion
915/433	~500 Kbps	~10s	~200 mW	Outdoor	No	Most common in UAV
Wi-Fi	>150 Mbps	~20s	~500 mW	Indoor	No	
Ours	>150 Mbps	~100s	~500 mW	Both	Yes	Modified Wi-Fi

Bold shows the best candidate in terms of one metric.

TABLE V
PARAMETER SETTINGS

Name	Parameter	Value	Unit
Body Mass	m	1.5	kg
Arm Length	l	0.15	m
Rotor Inertia	J_r	$1.04 \cdot 10^{-6}$	$\text{kg} \cdot \text{m}^2$
Drag Coefficient	d	$1.1 \cdot 10^{-6}$	Nms^2
Thrust Coefficient	d	$5.42 \cdot 10^{-5}$	Nms^2
Inertia on X Axis	I_x	$1.0 \cdot 10^{-2}$	$\text{kg} \cdot \text{m}^2$
Inertia on Y Axis	I_y	$1.5 \cdot 10^{-2}$	$\text{kg} \cdot \text{m}^2$
Inertia on Z Axis	I_z	$1.8 \cdot 10^{-2}$	$\text{kg} \cdot \text{m}^2$
Rotational Air Resistance Factor	K_r	$1.0 \cdot 10^{-2}$	Nms/rad
Translational Air Resistance Factor	K_t	$5.0 \cdot 10^{-4}$	Ns/m

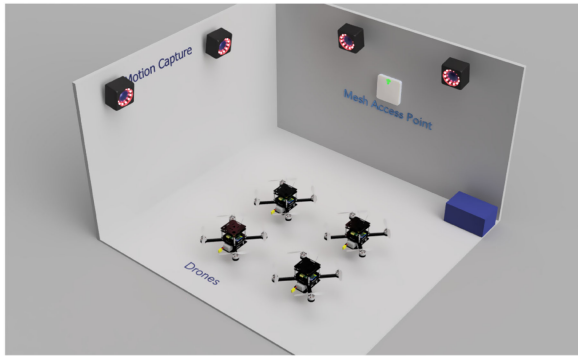


Fig. 7. UAV testbed system.

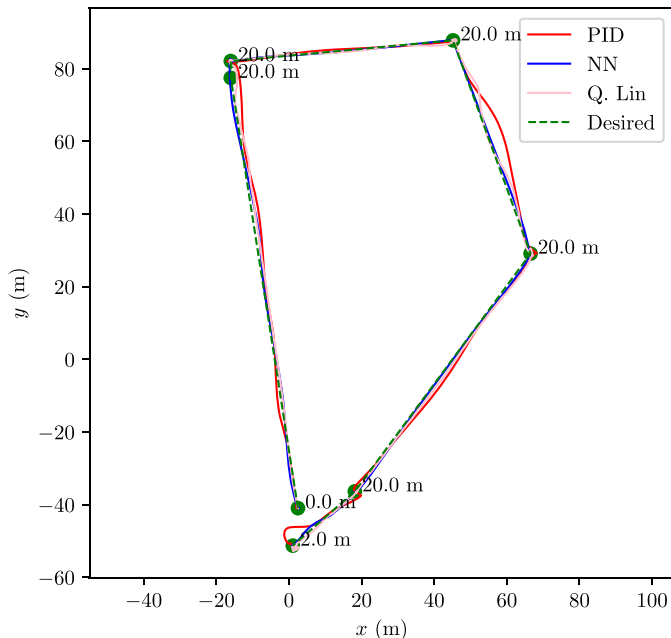


Fig. 8. Simulated flight trajectory. Labels indicate above sea-level altitude of the waypoint.

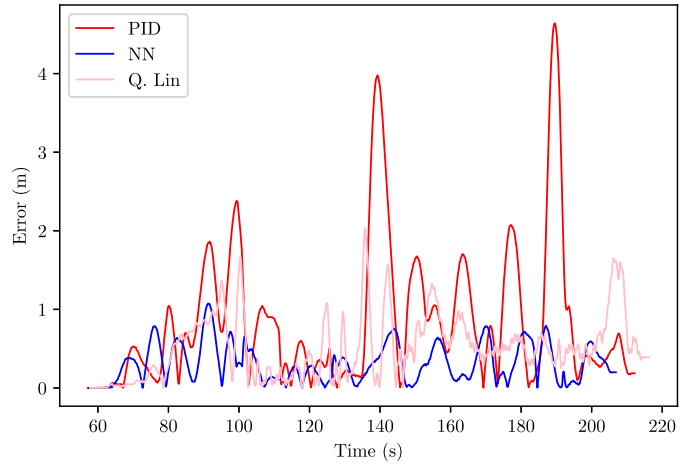


Fig. 9. Simulated trajectory errors generated by the three controllers during the flight.

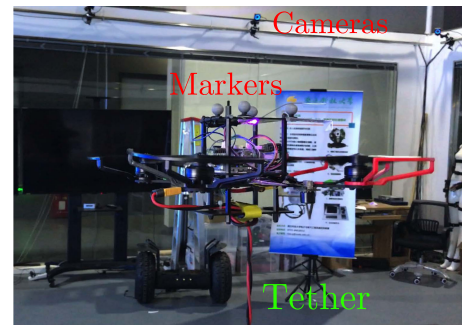


Fig. 10. Snapshot of the experiment setup.

nonrigid tether; and 2) body vibration caused by the motor and aerodynamics.

In both the tests, the vehicle was brought to hovering at the same altitude by using RC; then, the pilot releases all the control to the vehicle. At $t = 18$ s, we introduce a command via RC link by ordering the vehicle to rotate back-and-forth on the roll axis for 0.5 radians. All FCS parameters except NN engagement were untouched between the tests. Ground truth data were collected on an OptiTrack motion capture system installed in our test field (see Fig. 7).

C. Experiment Results

Fig. 11 shows the observed attitude values using the motion capture system. Immediately, we can see that the NN-based controller yields smaller errors than that of its PID counterpart, especially in the unbalanced roll axis. After the impulse at

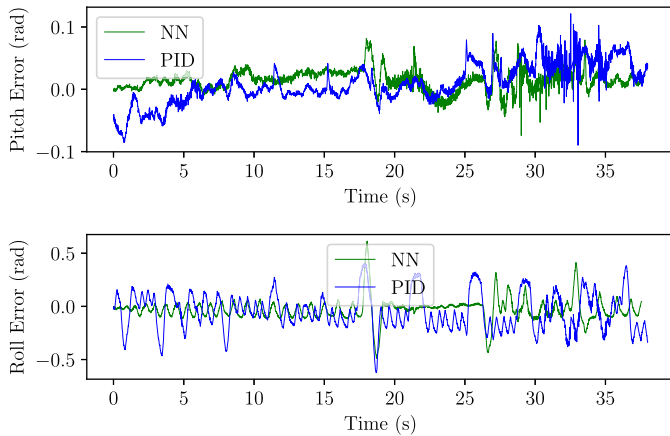


Fig. 11. Attitude errors of the quadrotor generated by two controllers.

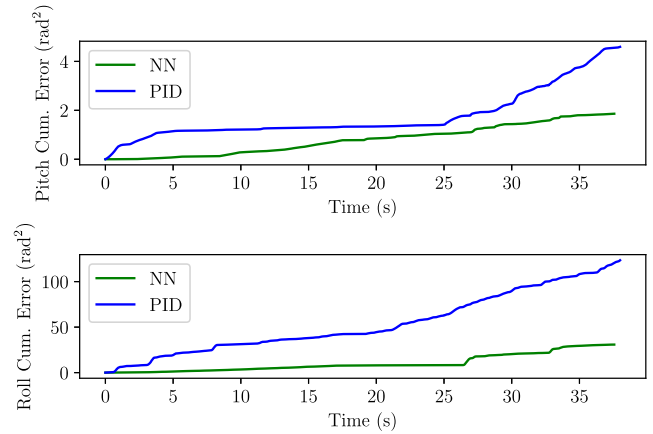


Fig. 13. Cumulative square attitude errors of the quadcopter.

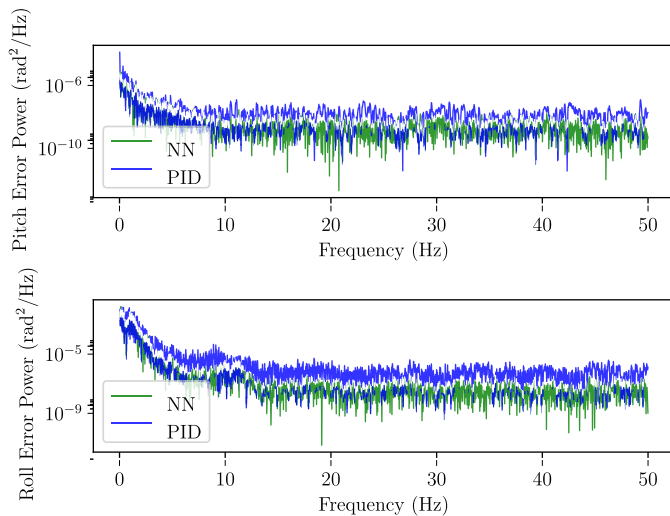


Fig. 12. Power spectra of the attitude errors during the flight.

$t = 18$ s, the NN-based controller returns to normal rapidly, while the PID controller encounters stability issues.

We can further confirm this difference in the attitude error spectrogram (see Fig. 12). In the pitch axis, the average error power ratio (NN:PID) is 0.250, while in the roll axis it is 0.692. Moreover, the NN-based controller is able to achieve an error reduction of around ten times as compared with the PID controller for frequencies ≥ 0.5 Hz. This significantly boosts vehicle stability and helps eliminating the errors induced by high-frequency noise in inertial guidance systems.

Fig. 13 shows the accumulated errors in both axes. It can be seen that the cumulative errors of the NN-based controller are less than those of the PID controller by more than half. Fig. 14 shows the open-loop position drift of the vehicle during the flight. In a wind-less environment, total drift directly correlates to the vehicle's ability to keep the thrust vertical. It can be observed that, thanks to the reduced attitude errors, the NN-based controller greatly outperformed the PID controller in the position drift test.

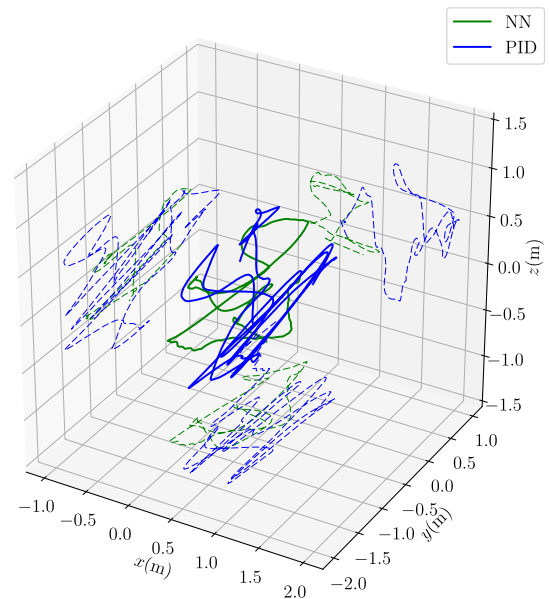


Fig. 14. Drift trajectory of the quadcopter. Dashed lines show the trajectory projected on xy , xz , and yz planes.

VI. CONCLUSION

In this paper, an adaptive control system combining an NN-enhanced dynamic inversion control scheme with a conventional PID controller was proposed for quadrotor UAVs. To improve the linear PID control system in challenging flight conditions, real-time SPNN-based dynamic inversion methods were developed to compensate the nonlinearity and external uncertainty of the system and improve the performance of quadrotor UAVs. To effectively implement and evaluate the control system, a set of hardware and software platforms were also developed accordingly. The simulation results showed that the proposed SPNN-based dynamic inversion control scheme outperformed the previous SPNN method as well as the conventional PID control in the presence of external disturbances. The experiment results demonstrated the superior performance of the proposed NN-based compensators in terms of tracking errors and open-loop position drifts in real-world scenarios. Our future work includes the enhancement of flight control systems with

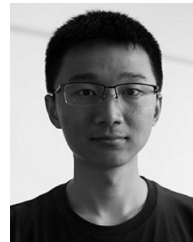
reinforcement learning schemes in the position planner to maximize the system adaptability in complex environments.

ACKNOWLEDGMENT

The authors would like to thank J. Sun for his help in testing the drones and setting up the motion capture system, C. Li for his help in the UAV's mechanical design, and anonymous reviewers for their valuable comments and suggestions.

REFERENCES

- [1] S. Islam, P. X. Liu, and A. E. Saddik, "Robust control of four-rotor unmanned aerial vehicle with disturbance uncertainty," *IEEE Trans. Ind. Electron.*, vol. 62, no. 3, pp. 1563–1571, Mar. 2015.
- [2] F. Chen, R. Jiang, K. Zhang, B. Jiang, and G. Tao, "Robust backstepping sliding-mode control and observer-based fault estimation for a quadrotor UAV," *IEEE Trans. Ind. Electron.*, vol. 63, no. 8, pp. 5044–5056, Aug. 2016.
- [3] Q. Lin, Z. H. Cai, Y. X. Wang, J. P. Yang, and L. F. Chen, "Adaptive flight control design for quadrotor UAV based on dynamic inversion and neural networks," in *Proc. 3rd Int. Conf. Instrum., Meas., Comput., Commun. Control*, Sep. 2013, pp. 1461–1466.
- [4] P. Poksawat, L. Wang, and A. Mohamed, "Automatic tuning of attitude control system for fixed-wing unmanned aerial vehicles," *IET Control Theory Appl.*, vol. 10, no. 17, pp. 2233–2242, Dec. 2016.
- [5] Z. Zhang and M. Cong, "Controlling quadrotors based on linear quadratic regulator," *Appl. Sci. Technol.*, vol. 5, pp. 38–42, 2011.
- [6] L. S. Brian and L. L. Frank, *Aircraft Control and Simulation*. Hoboken, NJ, USA: Wiley, 2003.
- [7] H. Gao, C. Liu, D. Guo, and J. Liu, "Fuzzy adaptive PD control for quadrotor helicopter," in *Proc. IEEE Int. Conf. Cyber Technol. Autom., Control, Intell. Syst. (CYBER)*, Jun. 2015, pp. 281–286.
- [8] S. Bouabdallah and R. Siegwart, "Backstepping and sliding-mode techniques applied to an indoor micro quadrotor," in *Proc. IEEE Int. Conf. Robot. Autom.*, Apr. 2005, pp. 2247–2252.
- [9] C. Li, G. Wang, Y. Fan, and Y. Li, "Adaptive RBF neural network controller design for SRM drives," in *Proc. 35th Chin. Control Conf.*, Jul. 2016, pp. 6092–6097.
- [10] L. Liu, Q. Geng, Q. Fei, and Q. Hu, "The longitudinal attitude control of UAV based on GPFN neural network," in *Proc. Int. Conf. Autom. Control Artif. Intell.*, Mar. 2012, pp. 2146–2149.
- [11] R. J. Adams and S. S. Banda, "Robust flight control design using dynamic inversion and structured singular value synthesis," *IEEE Trans. Control Syst. Technol.*, vol. 1, no. 2, pp. 80–92, Jun. 1993.
- [12] B. S. Kim and A. J. Calise, "Nonlinear flight control using neural networks," *J. Guid., Control, Dyn.*, vol. 20, no. 1, pp. 26–33, 1997.
- [13] J. V. R. Prasad, A. J. Calise, Y. Pei, and J. E. Corban, "Adaptive nonlinear controller synthesis and flight test evaluation on an unmanned helicopter," in *Proc. IEEE Int. Conf. Control Appl. (Cat. No. 99CH36328)*, Aug. 1999, pp. 137–142.
- [14] J. Leitner, A. Calise, and J. R. Prasad, "Analysis of adaptive neural networks for helicopter flight control," *J. Guid., Control, Dyn.*, vol. 20, no. 5, pp. 972–979, 1997.
- [15] X. Yang and L. M. Alvarez, "A neural-network based flight controller for UASs," *IFAC Proc. Volumes*, vol. 46, no. 10, pp. 164–171, 2013.
- [16] M. Fatan, B. L. Sefidgari, and A. V. Barenji, "An adaptive neuro PID for controlling the altitude of quadcopter robot," in *Proc. 18th Int. Conf. Methods Models Autom. Robot.*, Aug. 2013, pp. 662–665.
- [17] J. F. Shepherd III and K. Tumer, "Robust neuro-control for a micro quadrotor," in *Proc. 12th Annu. Conf. Genetic Evol. Comput. ACM*, 2010, pp. 1131–1138.
- [18] Y. Wang, Y. Chenxie, J. Tan, C. Wang, Y. Wang, and Y. Zhang, "Fuzzy radial basis function neural network PID control system for a quadrotor UAV based on particle swarm optimization," in *Proc. IEEE Int. Conf. Inf. Autom.*, Aug. 2015, pp. 2580–2585.
- [19] G. Shi and S. Yang, "Intelligent control of UAV with neuron-fuzzy approach under hierarchical architecture," in *Proc. 7th World Congr. Intell. Control Autom.*, Jun. 2008, pp. 5238–5243.
- [20] V. R. Puttige, S. Anavatti, and M. K. Samal, "Real-time validation of a dual neural network controller for a low-cost UAV," in *Proc. IEEE Int. Conf. Ind. Technol.*, Feb. 2009, pp. 1–6.
- [21] B. Tang, X. Fang, Z. Cai, and Y. Wang, "Reconfigurable flight control for UAV with multiple control effectors based on neural network and dynamic inversion," in *Proc. 2nd Int. Conf. Artif. Intell., Manage. Sci. Electron. Commerce*, Aug. 2011, pp. 2276–2281.
- [22] Lili and Z. Hua-min, "Neural network based adaptive dynamic inversion flight control system design," in *Proc. 2nd Int. Conf. Intell. Control Inf. Process.*, Jul. 2011, vol. 1, pp. 135–137.
- [23] L. Wang, Y. Chen, S. Li, and C. Liu, "2DOF manipulator tracking control based on fuzzy CMAC neural network dynamic inversion," in *Proc. Int. Conf. Intell. Comput. Integr. Syst.*, Oct. 2010, pp. 315–318.
- [24] T. Xiang, F. Jiang, Q. Hao, and W. Cong, "Adaptive flight control for quadrotor UAVs with dynamic inversion and neural networks," in *Proc. IEEE Int. Conf. Multisens. Fusion Integr. Intell. Syst.*, Sep. 2016, pp. 174–179.
- [25] B.-Y. Lee, H.-I. Lee, and M.-J. Tahk, "Analysis of adaptive control using on-line neural networks for a quadrotor UAV," in *Proc. 13th Int. Conf. Control, Autom. Syst.*, Oct. 2013, pp. 1840–1844.
- [26] M. G. Perhinschi, J. Burken, and G. Campa, "Comparison of different neural augmentations for the fault tolerant control laws of the WVU YF-22 model aircraft," in *Proc. 14th Mediterranean Conf. Control Autom.*, Jun. 2006, pp. 1–6.
- [27] T. Dierks and S. Jagannathan, "Output feedback control of a quadrotor UAV using neural networks," *IEEE Trans. Neural Netw.*, vol. 21, no. 1, pp. 50–66, Jan. 2010.
- [28] C. Powers, D. Mellinger, and V. Kumar, "Quadrotor kinematics and dynamics," in *Handbook of Unmanned Aerial Vehicles*. New York, NY, USA: Springer-Verlag, 2015, pp. 307–328.



Fan Jiang (M'19) received the B.S. degree in biomedical engineering from the Southern University of Science and Technology (SUSTech), Shenzhen, China, in 2017.

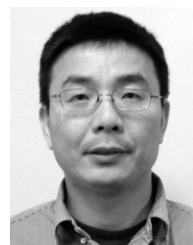
He is currently a Research Assistant with the Department of Computer Science and Engineering, SUSTech. His research interests include intelligent systems, robotics, and machine learning, especially in the design of robot simulation systems.



Farhad Pourpanah (M'17) received the Ph.D. degree in computational intelligence from University Science Malaysia (USM), George Town, Malaysia, in 2015.

He is currently an Associate Researcher with the College of Mathematics and Statistics, Shenzhen University (SZU), Shenzhen, China. Before joining SZU, he was a Postdoctoral Research Fellow with the Department of Computer Science and Engineering, Southern University of Science and Technology (SUSTech), Shenzhen.

His research interests include pattern recognition, evolutionary algorithms, and condition monitoring.



Qi Hao (M'05) received the B.E. and M.E. degrees in electrical and computer engineering from Shanghai Jiao Tong University, Shanghai, China, in 1994 and 1997, respectively, and the Ph.D. degree in electrical and computer engineering from Duke University, Durham, NC, USA, in 2006.

He received postdoctoral training with the Center for Visualization and Virtual Environment, University of Kentucky, Lexington, KY, USA. He was an Assistant Professor with the

Department of Electrical and Computer Engineering, University of Alabama, Tuscaloosa, AL, USA. He is currently an Associate Professor with the Department of Computer Science and Engineering, Southern University of Science and Technology (SUSTech), Shenzhen, China. His research interests include intelligent sensing, machine learning, and autonomous systems.